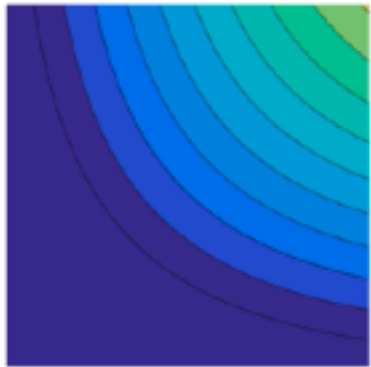




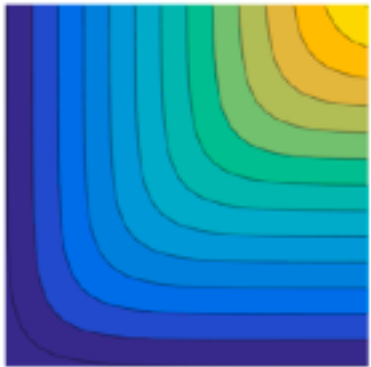
avoids underflow/overflow in Sinkhorn, enables smaller ϵ



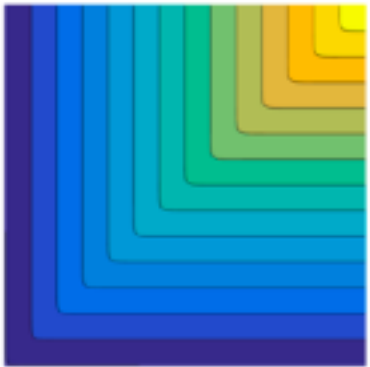
$$\varepsilon = 1$$



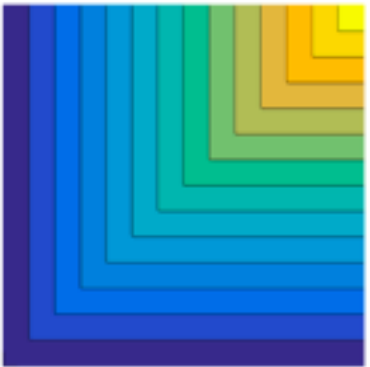
$$\varepsilon = 0.5$$



$$\varepsilon = 10^{-1}$$



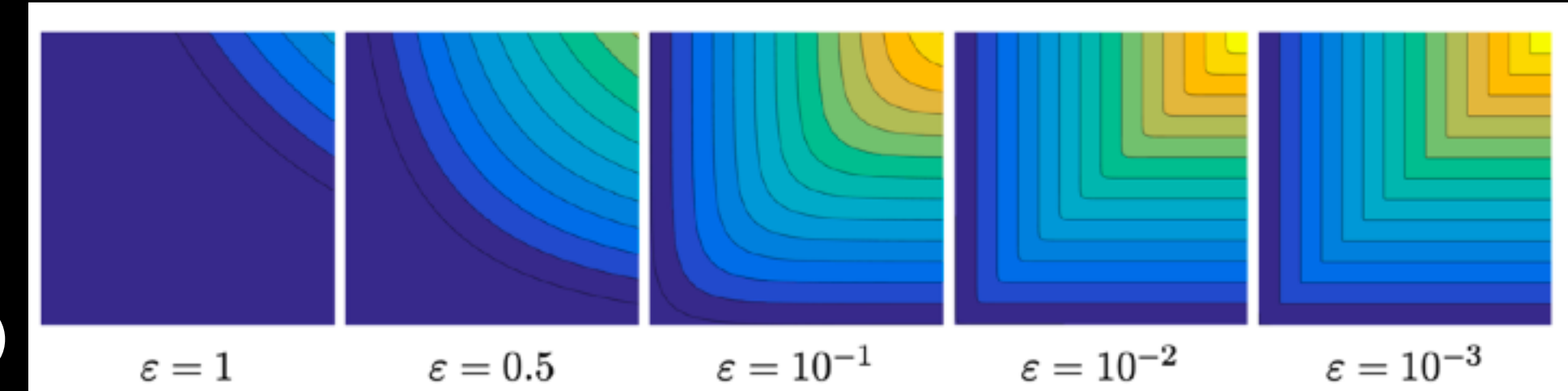
$$\varepsilon = 10^{-2}$$



$$\varepsilon = 10^{-3}$$

Log-Domain Sinkhorn

- Dropping (ℓ) for convenience, the log-domain sinkhorn updates are:
 - $\mathbf{f}_i = \varepsilon \log \mathbf{a}_i - \varepsilon \log(\mathbf{K} e^{\mathbf{g}/\varepsilon}) = \varepsilon \log \mathbf{a}_i - \varepsilon \log(\frac{1}{\varepsilon} \sum_j \exp\{\mathbf{g}_j - \mathbf{C}_{ij}\})$
 - $\mathbf{g}_j = \varepsilon \log \mathbf{b}_j - \varepsilon \log(\mathbf{K}^\top e^{\mathbf{f}/\varepsilon}) = \varepsilon \log \mathbf{b}_j - \varepsilon \log(\frac{1}{\varepsilon} \sum_i \exp\{\mathbf{f}_i - \mathbf{C}_{ij}\})$
- Define $\min_\varepsilon(\mathbf{z}) = -\varepsilon \log \sum_{i=1}^n e^{-\mathbf{z}_i/\varepsilon}$ (the soft-min operator, becomes min as $\varepsilon \rightarrow 0$). AKA log-sum-exp (LSE).
- We can use it to write:
 - $\mathbf{f}_i = \varepsilon \log \mathbf{a}_i + \min_\varepsilon(\mathbf{C}_{i,:} - \mathbf{g})$
 - $\mathbf{g}_j = \varepsilon \log \mathbf{b}_j + \min_\varepsilon(\mathbf{C}_{:,j} - \mathbf{f})$
- Note: if $\varepsilon \rightarrow 0$, boils down to enforcing constraints $\mathbf{f} \oplus \mathbf{g} \leq \mathbf{C} \implies \mathbf{f}_i \leq \min_j \mathbf{C}_{ij} - \mathbf{g}_j$ directly! (But doesn't converge)
- This can still be unstable if there is large spread in arguments to $\min_\varepsilon(\mathbf{z})$
- To avoid underflow, use the *log-sum-exp trick*: $\min_\varepsilon(\mathbf{z}) = m - \varepsilon \log \sum_{i=1}^n e^{-(\mathbf{z}_i - m)/\varepsilon}$, $m = \min \mathbf{z}$
- This how LSE is implemented in a numerically stable way in pytorch, numpy, etc
- **Takeaway:** operating in the log domain **avoids underflow/overflow in Sinkhorn, enables smaller ε**



Relations between EOT and OT